

Semantic Reconstruction of Software Intent

*A Cloud-Hosted Agentic AI Solution for the Study of
Requirement Traceability in Production-Grade Codebases*

CANDIDATE

Aarya Nanndaann Singh M N

BITS ID · 2024MT03013 | M.Tech Cloud Computing

Aqueralabs India Pvt Ltd, Bangalore

SUPERVISOR · EXAMINER · MENTOR

Mr. Hanumanthu Indrakanti — Director of Engineering

Mr. Deepan Kumar — Staff Engineer

Mr. Paresh Saxena — BITS Pilani Faculty Mentor

What we will cover today

01

The Problem

Intent erosion in production codebases
Why code-only is not enough

02

Architecture

Five-layer reference architecture
On AWS + MongoDB Atlas + OpenAI

03

Chunking, Embeddings, Retrieval

Section-aware chunks · 1536-d embeddings
Multi-query RAG with max-pool

04

IDE Integration & Cloud

VS Code / Cursor extension + loopback git agent
AWS Lambdas, EventBridge, Secrets Manager

05

Evaluation & Metrics

Functional checks · retrieval quality
Latency, cost and cognitive-load study

06

What's Next

Honest limitations · current improvements
Five directions for future work

SECTION 01 · CHAPTERS 1 & 3

The Problem.

Source code answers what. The interesting question is why.

Intent erosion: rationale gets lost over time

DEFINITION

The progressive loss of contextual rationale behind code as time passes — as authors leave the organisation and as artifacts become stale or scattered.

OBSERVABLE COST

- Long onboarding ramps for new joiners
- Recurring bugs that re-introduce previously fixed regressions
- Cognitive load of "context archaeology" through tickets, commits and chat
- Architectural decisions repeated because the why was never recorded

THE WHY QUESTIONS THAT SOURCE CODE ALONE CANNOT ANSWER

Why was this written this way?

Which requirement triggered this change?

Does the fix actually resolve the original defect?

How does this interact with previous decisions?

The reality at Aqueralabs: knowledge lives in silos

Atlassian Jira

Requirements
& incident tickets

Confluence

Design docs
& meeting notes

GitHub

Commits, PRs
& review threads

CI / CD

Build, validation
& deployment

THE DAILY JOIN PROBLEM

The rationale behind any non-trivial code change is split across three or more of these platforms.

New joiners and contributors perform the same mental joins repeatedly, often with incomplete information. We needed a semantic intermediary that ingests once, reconstructs intent on demand, and surfaces it inside the developer's existing tooling.

Formal statement and dissertation objectives

FORMAL STATEMENT

Given

$$(R, C, D, q) \rightarrow (E, S)$$

- R — requirement-bearing artifacts (Jira, Confluence)
- C — code-related artifacts (commits, PRs, reviews)
- D — developer's working tree, file f , range $\langle a, b \rangle$
- q — natural-language question from the developer
- E — explanation in plain language
- S — citation set, every claim attributable to it

OBJECTIVES

1

Characterise

intent erosion with concrete examples from production

2

Design

a semantic reconstruction framework — metadata + embeddings

3

Architect

a cloud-hosted agentic AI system combining RAG + multi-agent reasoning

4

Integrate

with VS Code and Cursor — no context-switching

5

Evaluate

functional correctness, operating cost and cognitive-load impact

Standing on four lines of prior work — and finding the gap

[1] Guo et al. (2024)

NLP for Requirements Traceability

Entity-aware preprocessing materially improves link quality vs. plain lexical search.

→ Adopted in the Jira normaliser and section-aware chunker.

[2] Wang, Zou, Wan et al. (2024)

Empirical Study on Req-to-Code Link Recovery

No single technique dominates; hybrid IR + neural approaches win.

→ Drove the structured-metadata + vector hybrid query design.

[3] Ali, Naganathan, Bork (2024)

RAG for Traceability

Retrieve, then generate with retrieved context as grounding.

→ Conceptual blueprint for the IntentEra reasoning engine.

[4][5] Ahmad et al. / Kanade et al.

Transformers for Code

Reusable embedding spaces for software engineering tasks without per-project tuning.

→ Justifies pluggable embedder; OpenAI text-emb-3-small today.

THE GAP

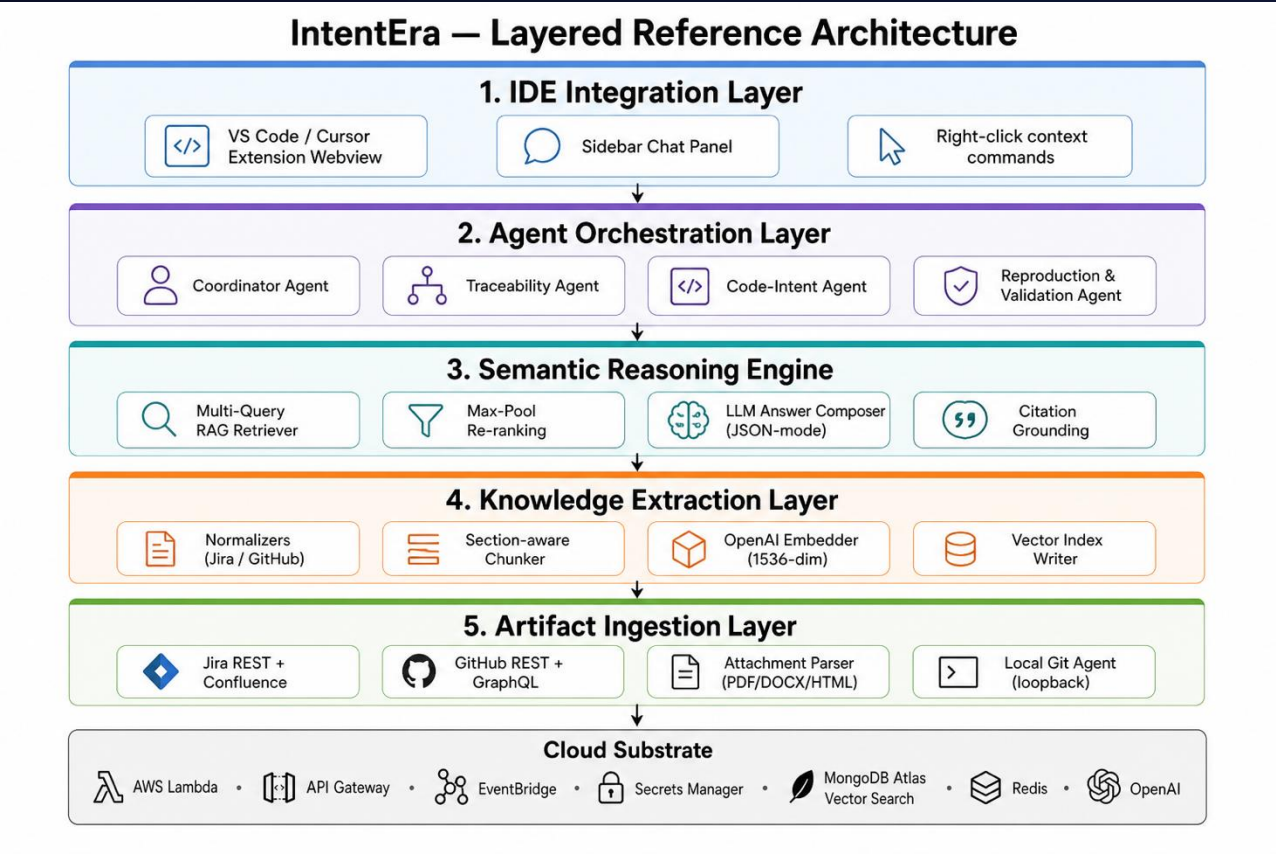
Almost no work in the intersection has been packaged as a deployable, IDE-integrated system that an engineering team can adopt incrementally.

SECTION 02 · CHAPTER 4

The Architecture.

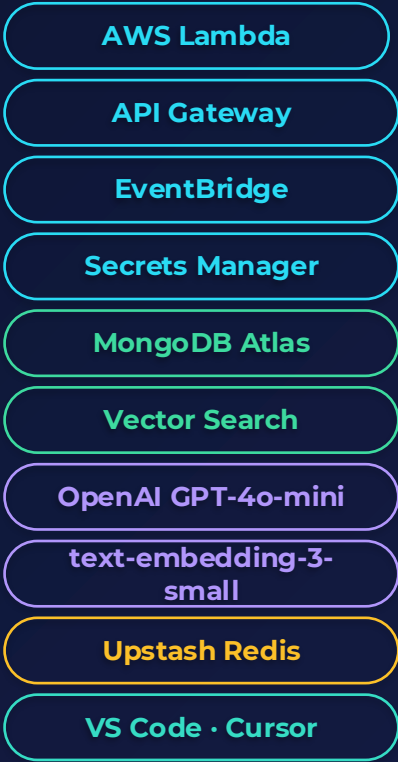
Five layers, source-neutral, idempotent — built for a small team's economics.

IntentEra at a glance



Five-layer reference architecture (Fig. 1, dissertation).

TECHNOLOGY STACK



FIVE LAYERS

- L1 Artifact Ingestion
- L2 Knowledge Extraction
- L3 Semantic Reasoning
- L4 Agent Orchestration
- L5 IDE Integration

The five layers, in one screen

L1 — Artifact Ingestion

Jira REST + Confluence · GitHub REST + GraphQL · Rate-limit aware · Exponential back-off

L2 — Knowledge Extraction

ADF → clean text · PDF / DOCX / HTML parsers · Section-aware chunker · OpenAI embeddings (1536-d)

L3 — Semantic Reasoning

Multi-query retriever · Parallel \$vectorSearch fan-out · Max-pool re-ranking · JSON-mode grounded answer

L4 — Agent Orchestration

Coordinator + Code-Intent (live) · Traceability (on demand) · Reproduction + Validation (scaffolded)

L5 — IDE Integration

Single VSIX (VS Code + Cursor) · Sidebar webview · Right-click commands · Loopback Local Git Agent

Five principles that shaped every decision



Layered separation

Ingestion, extraction, reasoning, orchestration and IDE are independently testable and replaceable.



Source neutrality

Jira, GitHub and future sources share the same embedding + vector-store pipeline. New source = fetcher + normaliser only.



Idempotence

Every chunk has a deterministic id
`sha256(source|entityKey|chunk Slot)` — re-runs are no-ops.



Strong grounding

The LLM never speaks without a context package. Citations not present in context are dropped before they reach the developer.



Cloud-native economics

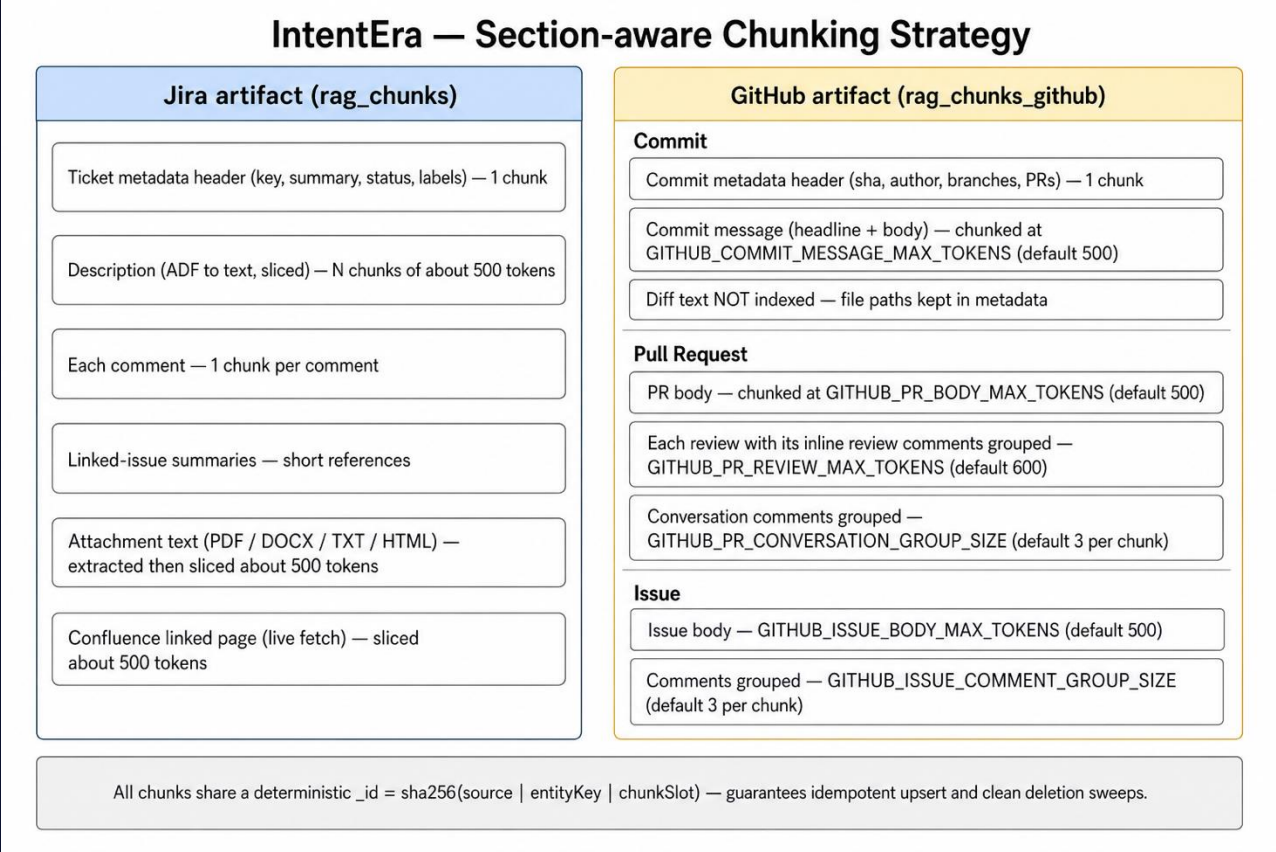
Pay-per-invocation Lambdas + free-tier Redis keep monthly cost in single-digit US-cents for bursty developer workloads.

SECTION 03 · CHAPTER 6

Chunking, Embeddings, Retrieval.

The reasoning engine, the heart of IntentEra.

Section-aware chunking — one idea per chunk



Idempotent ids + 1536-dim embeddings

DETERMINISTIC CHUNK ID

`_id = sha256( source | entityKey | chunkSlot )`

Re-ingesting the same artifact produces the same ids — MongoDB upsert is a no-op when nothing has changed, a consistent overwrite when it has.

1,536

EMBEDDING DIMENSIONS

OpenAI text-embedding-3-small — shared across natural language and code-adjacent artifacts.

1× class

PLUGGABLE EMBEDDER

Swap to a code-specialised model by changing one constructor — no other code touches.

Batched

SINGLE ROUND-TRIP

All query embeddings batched in one OpenAI call. Exponential back-off on transient failures.

→ *Hourly incremental sync re-embeds only what changed since the previous run — the dominant external cost lever, controlled by design.*

Two collections, one cluster, hybrid filters

rag_chunks

Source: Jira

_id · ticketKey · projectKey · type
(metadata / description / comment / attachment)
chunkIndex · text · embedding[1536]

rag_chunks_github

Source: GitHub

_id · entityKey (e.g. commit:<sha>) · entityType
repoFullName · branches · prNumber
chunkSlot · text · embedding[1536]

ATLAS CLUSTER

M10 · Three-node Replica Set

ap-south-2 (Hyderabad)

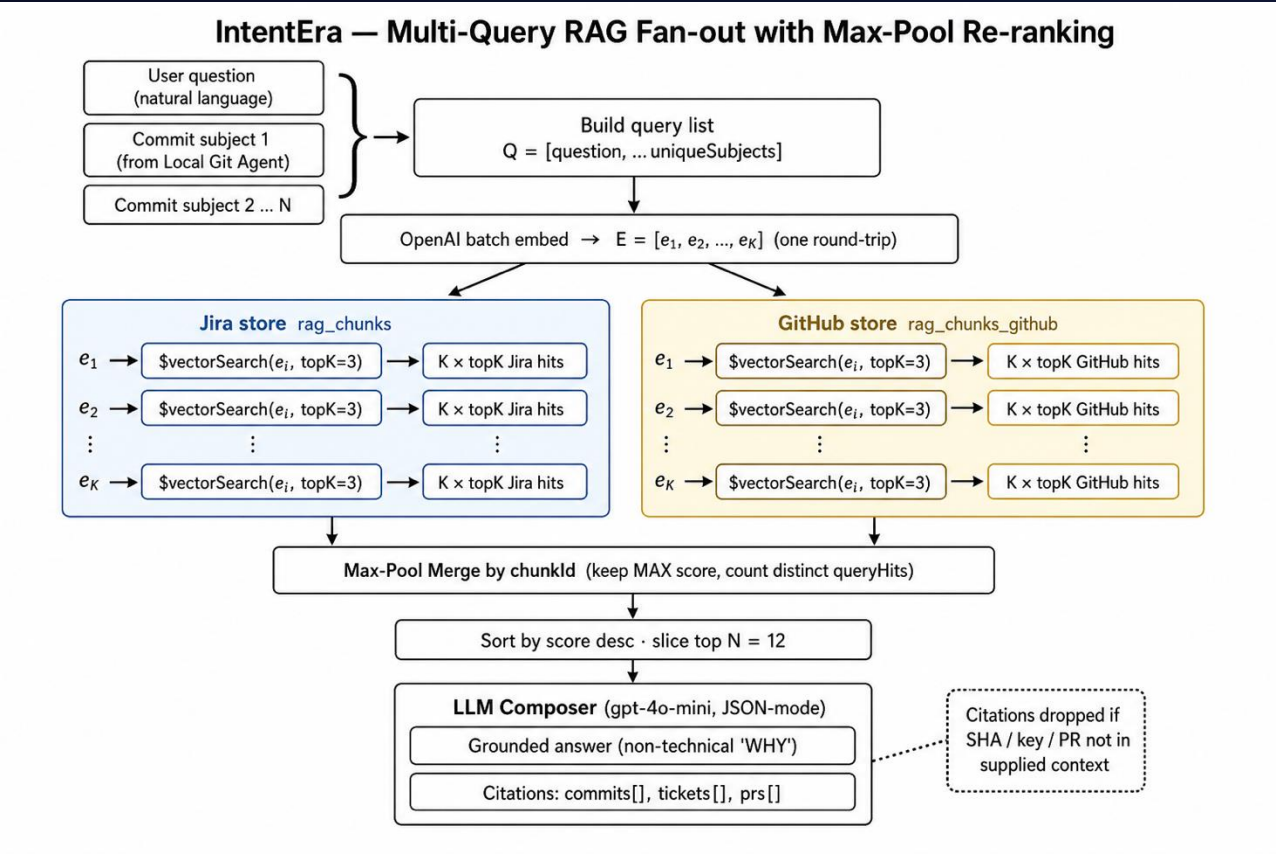
Two vector indexes — vector_index and vector_index_github — over the 1536-dim embedding field, plus per-source filter fields for hybrid IR + neural queries.

WHY THE REGION SPLIT

AWS workload in ap-south-1 (Mumbai) — proximity to the team.

Atlas in ap-south-2 (Hyderabad) — additional redundancy without trans-oceanic latency on every query.

Multi-query retriever — the heart of the engine



Multi-query RAG fan-out (Fig. 33, dissertation).

ALGORITHM — FOUR STEPS

- 01**
Dedupe & batch
Concatenate question + unique local commit subjects → list Q.
- 02**
Embed once
All K queries embedded in a single OpenAI call.
- 03**
Parallel \$vectorSearch
Each embedding → both Atlas collections, top-K = 3.
- 04**
Max-pool merge
Group by chunkId, keep max score, sort, slice to top N (= 12).

Why — and why not the obvious alternatives max-pool

Average score across queries

✗ Rejected

Dilutes a chunk that is decisively relevant to one query but only weakly relevant to another — exactly the signal we want to preserve.

Concatenate queries into one string

✗ Rejected

Conflates orthogonal signals into a single embedding, degrading retrieval quality for every individual query.

Max-pool by chunkId ← the choice

✓ Adopted

Keep the maximum score across queries for each chunk. A ticket whose title closely matches one commit subject is promoted into the top of the merged list, even when its match against the user's question is moderate.

→ *Max-pool is essentially a soft ensemble — it lets the strongest evidence per chunk speak, regardless of which query found it.*

Three-layer defence against hallucination

01

JSON-Mode Prompt

LLM (OpenAI gpt-4o-mini) is forced into JSON-mode and instructed to emit a non-technical "why" paragraph plus three citation lists — commits, tickets, PRs.

02

Post-Filter Citations

Any commit SHA, ticket key or PR number that does NOT appear in the supplied context is silently dropped before the response leaves the Lambda.

03

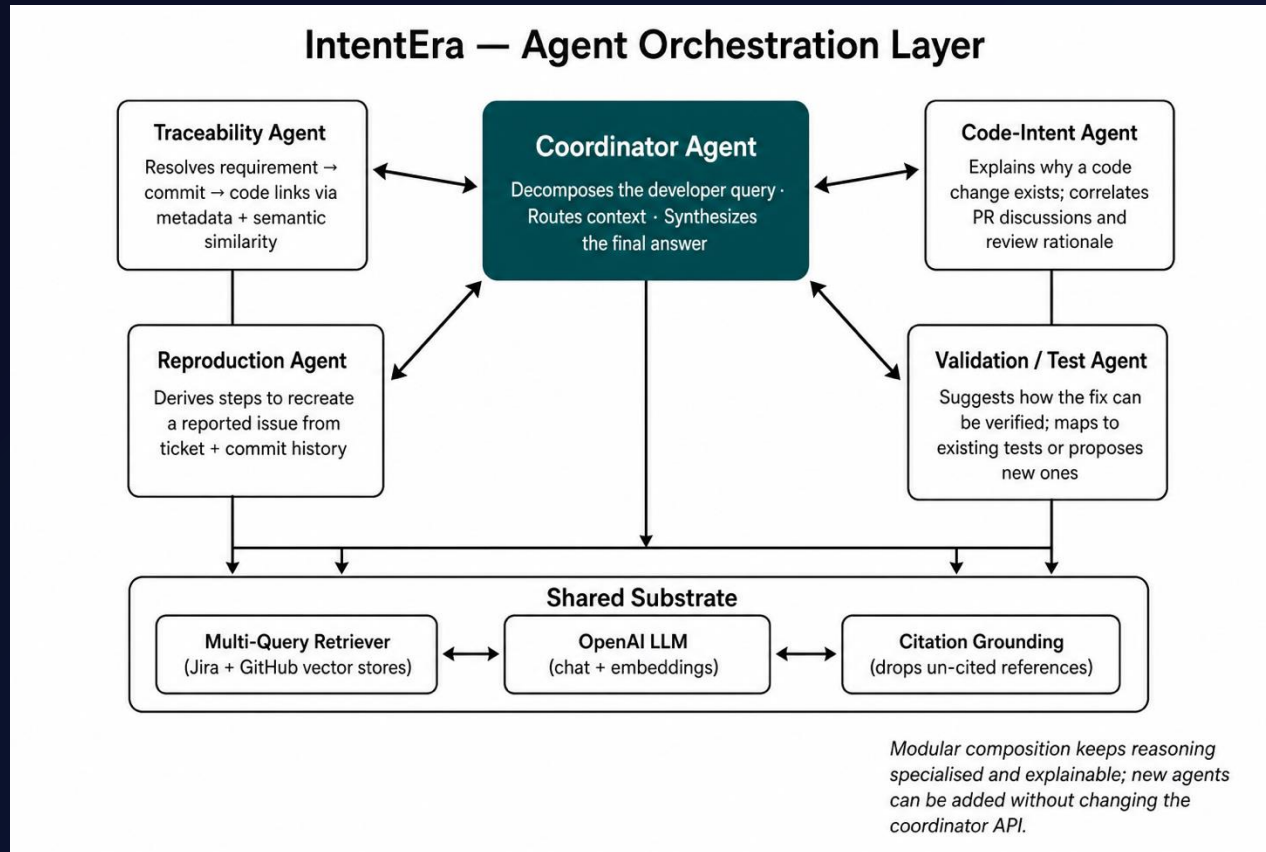
Clickable in the IDE

Citations render as clickable references inside the chat panel — the developer can verify each source in one click.

RESPONSE SHAPE (TRUNCATED) — APPENDIX B.5

```
{
  "answer": "In plain English: the retry was added because the SSO ...",
  "citations": {
    "commits": [ { "sha": "abc1234567...", "subject": "fix(auth): ..." } ],
    "tickets": [ { "key": "PROJ-123", "summary": "Intermittent SSO ..." } ],
    "prs": [ { "repo": "acme/platform", "number": 451 } ]
  },
  "retrieved": { "jiraCount": 5, "githubCount": 4, "mergedCount": 9 }
}
```

Specialised agents under one coordinator



Coordinator routes context to specialised agents (Fig. 34).

AGENT ROSTER

LIVE IN PRODUCTION

● **Coordinator**
Routes context · synthesises responses

● **Code-Intent**
Composes the "why" explanation

ON DEMAND

● **Traceability**
Resolves "which requirement does this implement?"

SCAFFOLDED — FUTURE WORK

● **Reproduction**
Derives repro steps from tickets + commits

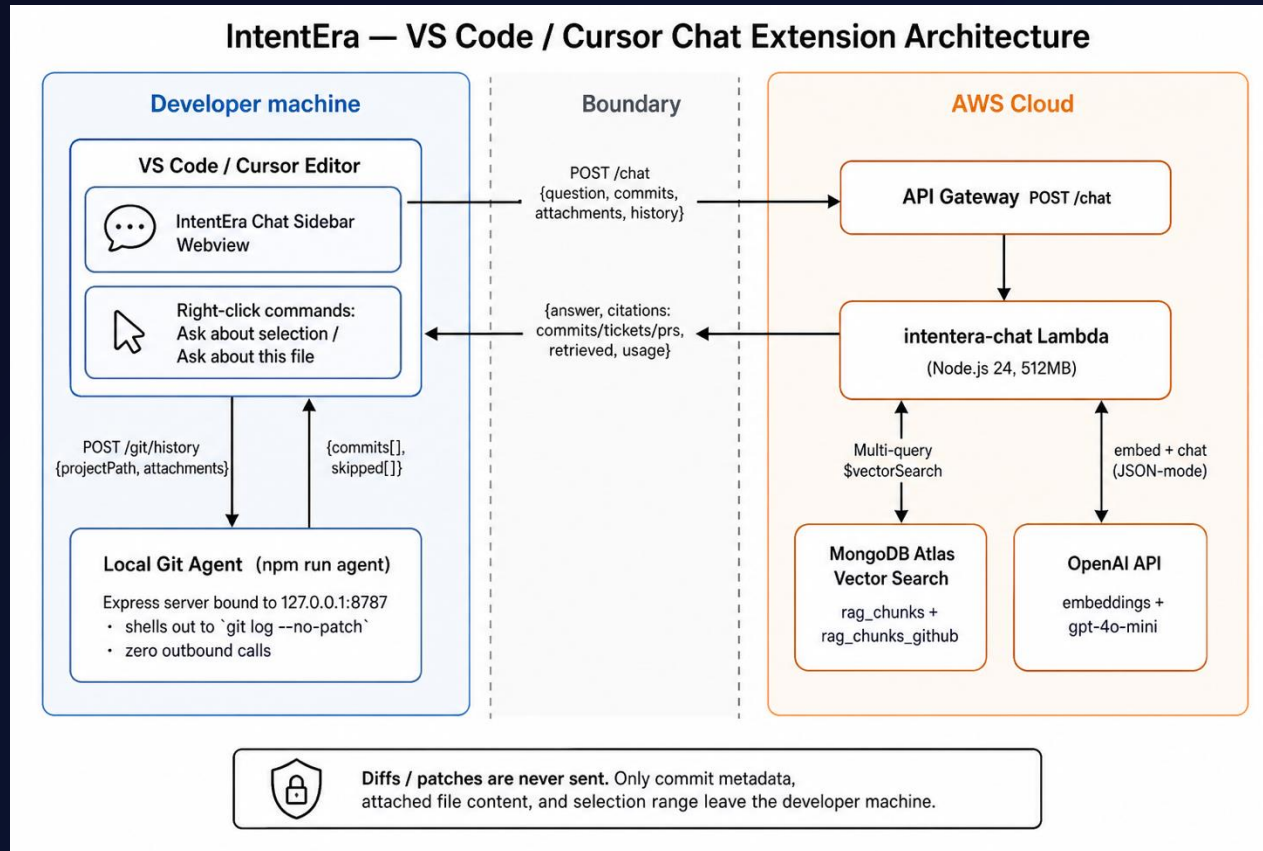
● **Validation / Test**
Suggests how to verify the fix

SECTION 04 · CHAPTERS 5 & 7

IDE Integration & Cloud.

Where the developer experiences IntentEra — and how AWS makes it cheap.

Three pieces — and a deliberate trust boundary



Three-piece chat extension architecture (Fig. 35).

01 · VSIX Extension

Single artifact installs into both VS Code and Cursor (same Code-OSS extension API). Sidebar webview + right-click commands.

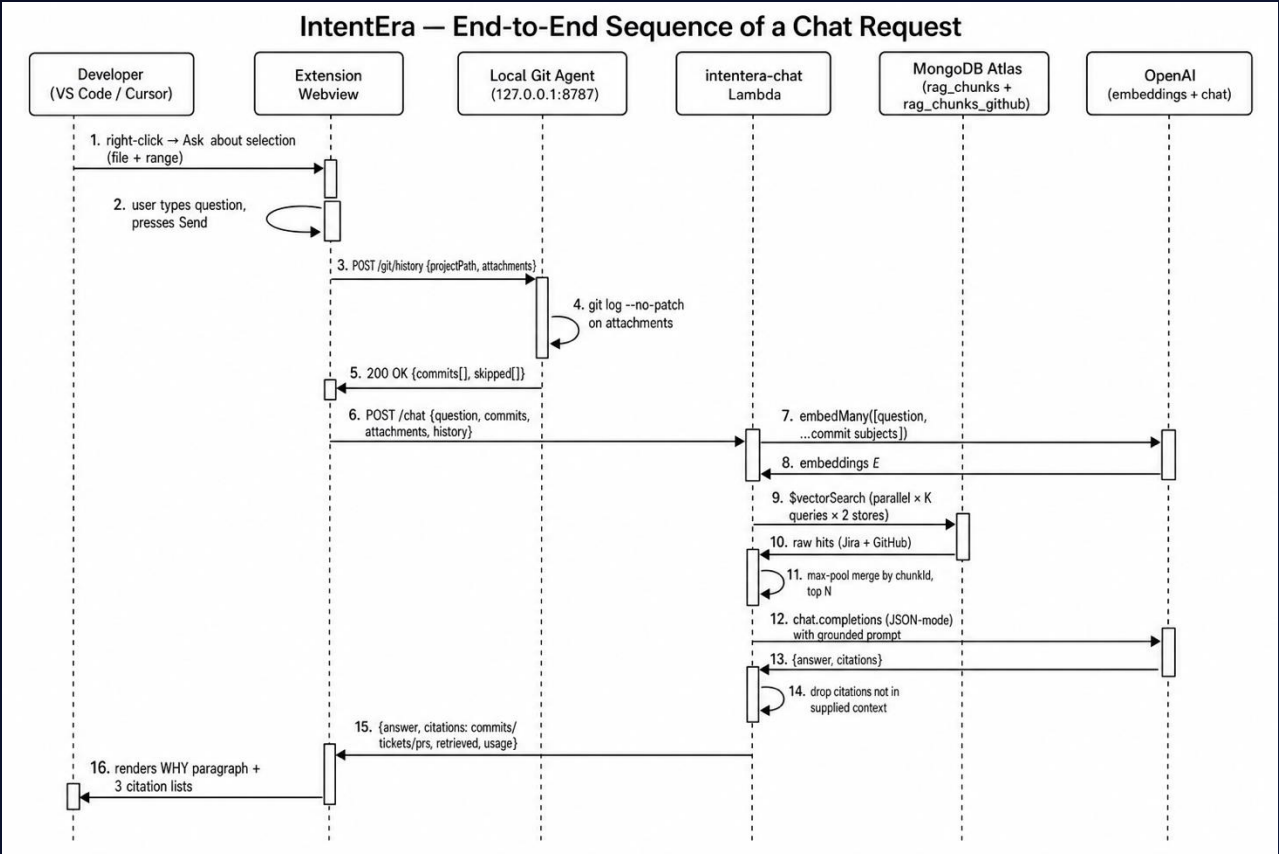
02 · Local Git Agent

Tiny Express server on 127.0.0.1:8787 — loopback only. Uses git log -no-patch → metadata only, never diffs. Allowlistable project roots.

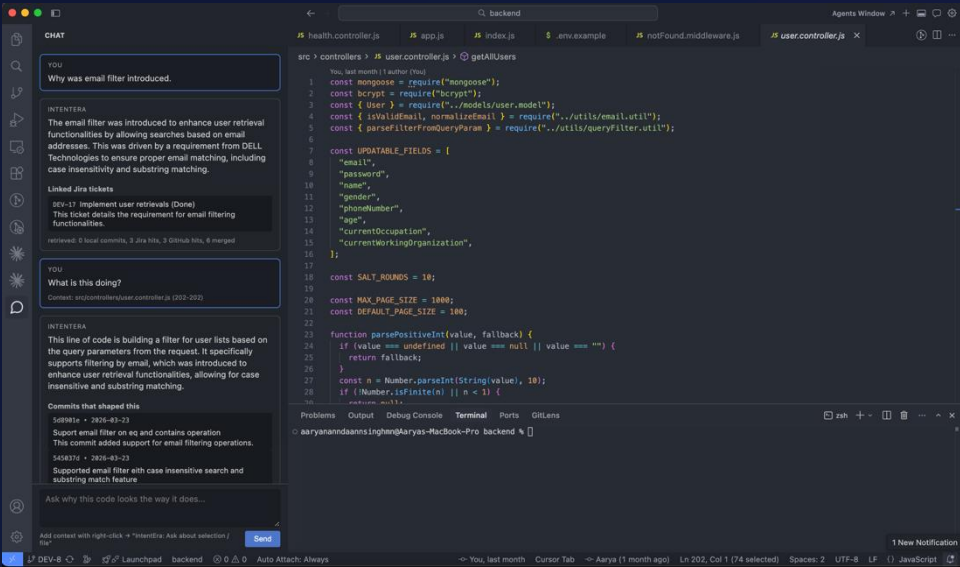
03 · Cloud Chat Lambda

Called via API Gateway. Runs the multi-query RAG pipeline + JSON-mode answer composition + citation post-filtering.

From right-click to grounded answer

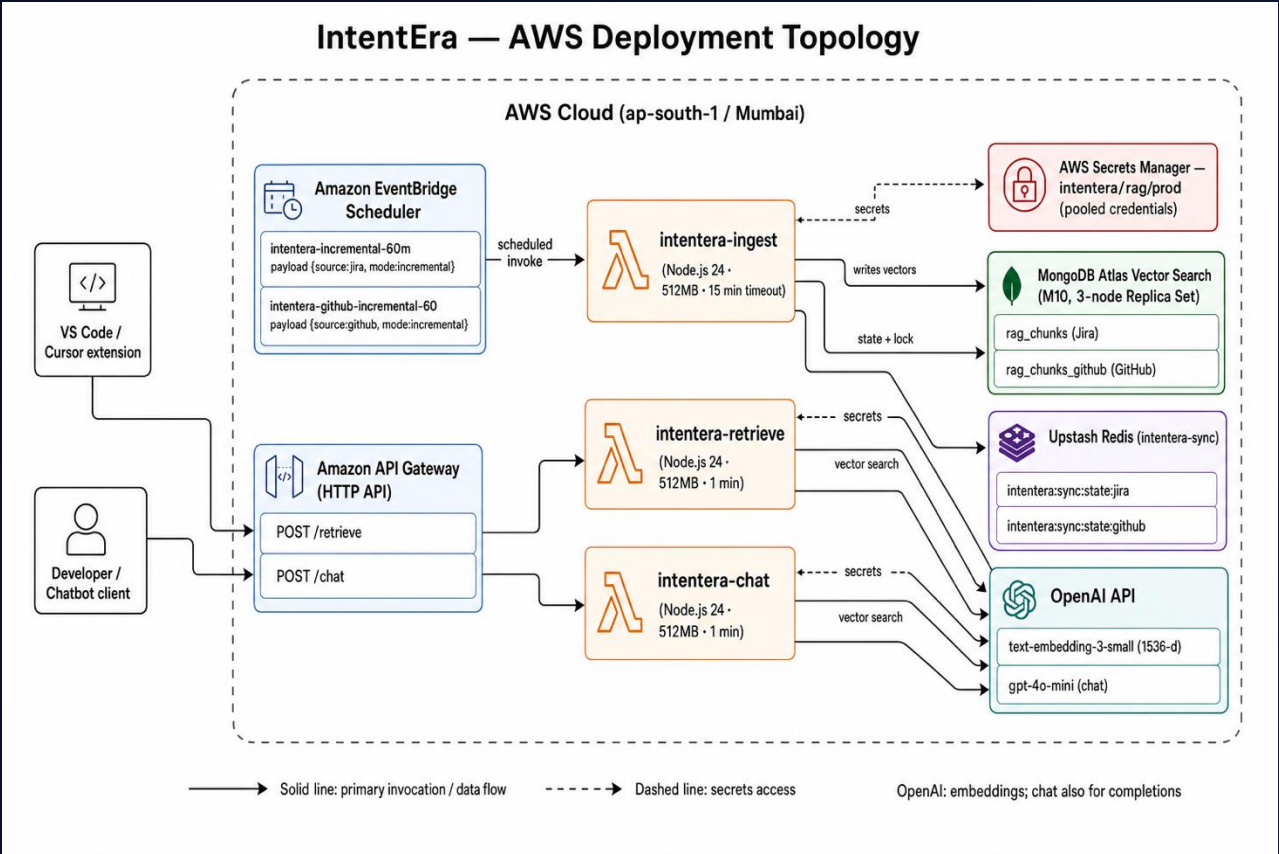


UML-style sequence (Fig. 36).

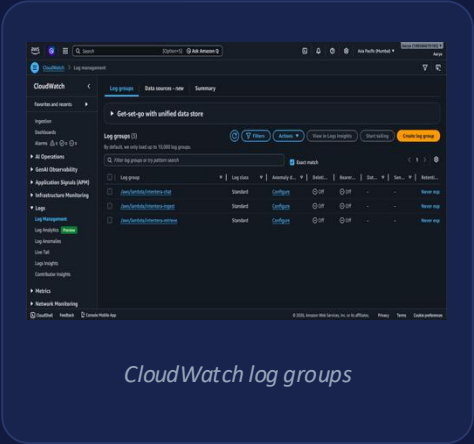
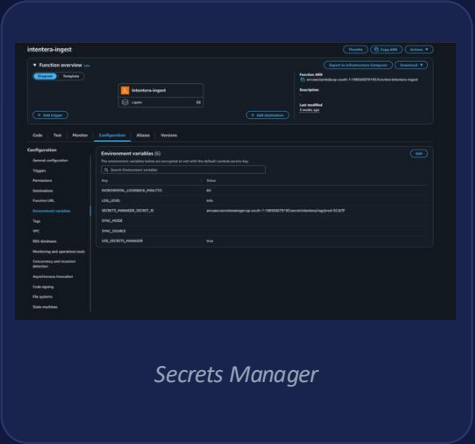
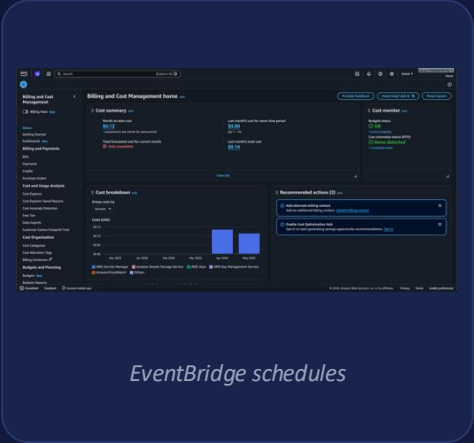
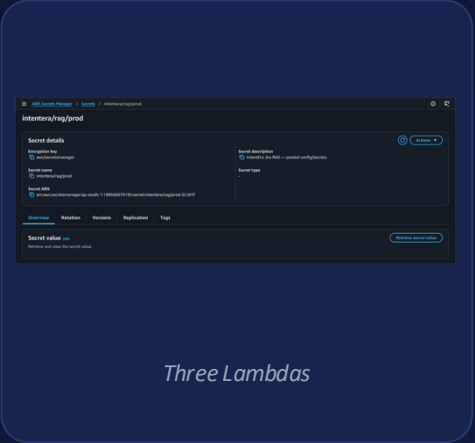


Cursor chat panel: "Why was the email filter introduced?"

Production deployment on AWS



EventBridge → Ingest Lambda; API Gateway → Retrieve & Chat (Fig. 2).



SECTION 05 · CHAPTER 8

Evaluation & Metrics.

Functional, retrieval, latency, cost — and what three engineers actually felt.

10 of 10 functional checks pass

#01 Lambda cold-start under 5 s P	#06 POST /retrieve returns ranked hits + metadata P
#02 Full Jira import populates rag_chunks P	#07 POST /chat returns grounded answer with citations P
#03 Full GitHub import populates rag_chunks_github P	#08 Citations not in context are dropped P
#04 Incremental sync respects 60-min schedule P	#09 Local git agent rejects non-loopback callers P
#05 Re-running incremental is idempotent P	#10 Extension banner reflects agent + Lambda health P*

P — minor observation: extension banner wording was reworted twice during testing for non-author readability.*

Retrieval quality and cognitive-load impact

15 / 15 QUESTIONS
WITH ≥ 1 GOLD
ARTIFACT IN
TOP-12

Curated from real Slack threads in the employing organisation, supervisor-graded.

≤ 3 POSITION OF THE FIRST
GOLD ARTIFACT
(MEDIAN)

In the majority of cases the first gold artifact appeared in the top three merged hits.

3 / 3 ENGINEERS SHOWED
REDUCED CONTEXT-
SWITCHES

30-min sessions on unfamiliar repos with vs. without the extension. Self-reported confidence rose because citations are checkable.

MAX-POOL RE-RANKING IN THE WILD

Max-pool behaved as designed.

Artifacts that scored only moderately on the user's question but very strongly on a specific commit subject — for example a Jira ticket whose title closely matched a commit subject — were reliably promoted into the top of the merged list.

A formal precision-recall evaluation against a larger dataset and a NASA-TLX cognitive-load study are identified as future work.

Latency and cost — viable for a small team

< 5 s

TYPICAL END-TO-END CHAT REQUEST

Measured across 50 real interactions — right-click to rendered answer. Well under the 60 s Lambda timeout. LLM call dominates.

single-digit ¢

AWS SPEND PER MONTH

Lambdas + Secrets Manager + EventBridge. Atlas M10 is the lower bound of the dedicated tier.

K × 2

ATLAS \$VECTORSEARCH OPS PER CHAT

K = #queries (1 + per local commit subject); × #stores (Jira + GitHub). Negligible per query, parallelised across stores.

2 × OpenAI

EXTERNAL LLM CALLS PER CHAT

One batched embedding round-trip + one chat completion. Embeddings are amortised across all queries in the request.

Incremental sync re-embeds only what changed → OpenAI cost scales with delta, not corpus size.

SECTION 06 · CHAPTERS 9 & 10

What's Next.

Honest limitations, what we shipped, and where the framework goes from here.

Challenges and limitations — documented honestly

NL → code gap

Requirements are informal, code is structured. Multi-query retriever helps; abstract requirements remain edge cases.

Noisy artifacts

"minor fix" commits, empty PRs. Mitigated by preserving metadata + ingesting reviewer comments as their own chunks.

Missing traceability

Many commits don't reference a ticket. Combination of metadata + semantic similarity, with hit counts exposed.

Source heterogeneity

Each platform has its own format and rate-limits. Per-source normalisers and Redis state keep the reasoning core clean.

LLM hallucination

Three-layer defence: JSON-mode + post-filter + clickable citations. Cannot guarantee zero, substantially reduced.

Cold start

Mongo / Redis / OpenAI clients are cached across warm invocations. Provisioned concurrency is available if needed.

Scale & scope

2 source families today, managed-cloud only, no benchmarks at 100k+ artifacts yet. Designed for incremental growth.

What's improved since the midterm - already in production



Section-aware chunker

Replaces uniform-length splits with one-idea-per-chunk windows. Removed cross-boundary loss of key claims.



JSON-mode + post-filter grounding

LLM forced into JSON-mode; uncited identifiers stripped before reaching the developer. Eliminates fabricated SHAs.



Max-pool re-ranking

Single-batch query embedding + parallel \$vectorSearch + max-pool merge replaces single-query nearest-neighbour search.



Loopback Local Git Agent

Replaces sending diffs/patches to the cloud. Only metadata leaves the machine; agent binds to 127.0.0.1.



Single VSIX, two IDEs

One artifact installs into both VS Code and Cursor. Same right-click commands, same sidebar everywhere.

Five directions worth pursuing next

01

Code Evolution Graph

Index the repository's call graph as a first-class artifact — answer questions that depend on cross-file structure, not only textual similarity.

02

Explainable per-citation Confidence

Surface the per-query hit counts already computed by the retriever — let developers calibrate trust per cited artifact.

03

Regression-Test Suggestion Agent

Extension of the Validation/Test agent — propose specific tests to add after a fix, grounded in the same artifacts the chat answer cites.

04

Cross-Repository Dependency Tracing

Ingest multiple repositories and express their dependencies in the vector index — addresses platform-and-service split monorepos.

05

Knowledge-Graph Semantic Memory

Layer an explicit knowledge graph on top of the vector store — enables path-based and temporal questions that vector similarity alone cannot support.

**Implementation rationale,
often assumed to be irretrievably lost
as software evolves,
can in fact be computationally reconstructed.**

IntentEra demonstrates that intent-aware development environments are not only feasible but practical — and that traceability can be an active form of cognitive augmentation, not a documentation aspiration.

THANK YOU

Questions & Discussion

With thanks to my supervisor, examiner and faculty mentor.

AARYA NANNDAANN SINGH M N

BITS ID · 2024MT03013

M.Tech Cloud Computing

WILP Dissertation CCZG628T · May 2026